

Musterlösung zur Vorbereitungsklausur Programmierung II

Daniel Appenmaier, 27. Juli 2026

Aufgabe 1 (19)

Klasse *Performance*

```
public record Performance(String bandName, LocalTime startTime, MusicGenre genre) // 1,5
    implements Comparable<Performance> { // 0,5

    @Override
    public int compareTo(Performance other) { // 0,5
        return startTime.compareTo(other.startTime()); // 1,5
    } // 2

} // 4
```

Klasse *Festival*

```
public record Festival(String name, Map<Performance, Stage> schedule) { // 1

    public void addPerformance(Performance performance, Stage stage) // 0,5
        throws DuplicatePerformanceException { // 0,5
        if (schedule.containsKey(performance)) { // 1
            throw new DuplicatePerformanceException(); // 1
        }
        schedule.put(performance, stage); // 1
    } // 4

    public Optional<Stage> getStageByBandName(String bandName) { // 0,5
        for (Entry<Performance, Stage> entry : schedule.entrySet()) { // 1
            if (entry.getKey().bandName().equals(bandName)) { // 1,5
                return Optional.of(entry.getValue()); // 1,5
            }
        }
        return Optional.empty(); // 1
    } // 5,5

    public List<Performance> getPerformancesByGenre(MusicGenre genre) { // 0,5
        List<Performance> result = new ArrayList<>(); // 0,5
        for (Performance performance : schedule.keySet()) { // 1
            if (performance.genre() == genre) { // 1
                result.add(performance); // 0,5
            }
        }
        Collections.sort(result); // 0,5
        return result; // 0,5
    } // 4,5

} // 15
```

Aufgabe 2 (17)

Klasse *FestivalTest*

```
public class FestivalTest { // 0,5

    private Festival festival; // 0,5
    @Mock // 0,25
    private Performance performanceA; // 0,25
    @Mock // 0,25
    private Performance performanceB; // 0,25
    @Mock // 0,25
    private Stage stageLarge; // 0,25

    @BeforeEach // 0,5
    void setUp() { // 0,5
        MockitoAnnotations.openMocks(this); // 1
        festival = new Festival("Sommer Open Air", new HashMap<>()); // 1

        festival.addPerformance(performanceA, stageLarge); // 0,5
        when(performanceA.startTime()).thenReturn(LocalTime.of(12, 0)); // 1
        when(performanceA.genre()).thenReturn(MusicGenre.ROCK); // 1

        festival.addPerformance(performanceB, stageLarge); // 0,5
        when(performanceB.genre()).thenReturn(MusicGenre.POP); // 1
        when(performanceB.startTime()).thenReturn(LocalTime.of(9, 0)); // 1
    } // 8

    @Test // 0,5
    void addPerformanceDuplicateThrows() { // 0,5
        assertThrows(DuplicatePerformanceException.class,
            () -> festival.addPerformance(performanceA, stageLarge)); // 1,5
    } // 2,5

    @Test // 0,5
    void getPerformancesByGenreReturnsSortedByStartTime() { // 0,5
        List<Performance> result =
            festival.getPerformancesByGenre(MusicGenre.ROCK); // 1
        assertEquals(1, result.size()); // 1
        assertEquals(performanceA, result.get(0)); // 1
    } // 4

} // 17
```

Aufgabe 3 (16)

Klasse *FestivalQueries*

```
public record FestivalQueries(Map<Performance, Stage> schedule) { // 1

    public List<String> getAllBandNamesSorted() { // 0,5
        return schedule.keySet().stream() // 1,5
            .map(Performance::bandName) // 0,5
            .distinct() // 0,5
            .sorted() // 0,5
            .toList(); // 0,5
    } // 4

    public List<Stage> getAllStagesSortedByName() { // 0,5
        return schedule.values().stream() // 1,5
            .distinct() // 0,5
            .sorted((s1, s2) -> s1.name().compareTo(s2.name())) // 1
            .toList(); // 0,5
    } // 4

    public Map<LocalTime, List<Performance>> getPerformancesByStartTime() { // 0,5
        return schedule.keySet().stream() // 1,5
            .collect(Collectors.groupingBy(Performance::startTime)); // 1,5
    } // 3,5

    public long numberOfStagesBySize(StageSize size) { // 0,5
        return schedule.values().stream() // 1,5
            .filter(s -> s.size() == size) // 1
            .count(); // 0,5
    } // 3,5

} // 16
```